

Third-Party Transactional Email Research

RelayForge Operations Platform · TCW-71 · Technical evaluation and integration blueprint

Decision: Amazon SES is the primary recommendation; Mailgun and Twilio SendGrid are the two implementation-ready alternatives.

Project Overview

RelayForge is a multi-tenant e-commerce operations platform for returns, warranty claims, refunds, shipping exceptions, and operator/customer case workflows. The requested integration introduces provider-neutral transactional customer email without changing existing synchronous API, PostgreSQL outbox, or BullMQ worker boundaries.

Scope is limited to customer account verification, return authorization, shipping-label-ready, refund-status, warranty-case, and human-agent-message notifications. Bulk or marketing email is explicitly excluded.

Jira issue: [TCW-71](#) — Research a transactional email delivery integration for RelayForge.

Authoritative linked source: [RelayForge Transactional Email Integration — Research Source Pack](#) .

Jira project description: Not specified in source. Issue attachments: none. Linked documents outside the approved research folder: none.

Requirement attribution: business requirements and acceptance criteria come from TCW-71; architecture/API, performance, security, compliance and operating constraints are corroborated by the linked RelayForge Transactional Email Integration Research Source Pack. Repository files establish the actual technology and deployment context.

Feature Requirements

Functional requirements

- Deliver six tenant-scoped transactional notifications: account verification, return authorization, shipping label ready, refund status, warranty case, and human-agent messages. Source: TCW-71 business requirements; source pack, Functional Requirements.
- Render code-reviewed, versioned HTML and plain-text templates in en and fr-CA with merchant-specific branding. Source: TCW-71 business requirements; source pack, Functional Requirements.

- Support approximately 180 merchant identities, verified sending domains, isolated reputation/configuration, individual disablement, safe nonproduction testing, opaque correlation metadata, and attachments or expiring signed S3 links. Source: TCW-71 business requirements; source pack, Functional Requirements.
- Normalize accepted, delivered, deferred, bounced, complained, and rejected provider events into searchable, tenant-scoped case-audit history. Source: TCW-71 business requirements; source pack, Functional Requirements.

Non-functional requirements

- Sustain 35,000 messages per business day, 25 submissions/second normally, and 90 submissions/second for ten-minute surges; keep synchronous API acceptance p95 below 150 ms and provider handoff under 60 seconds p95 / five minutes p99. Source: TCW-71; source pack, Performance and Reliability Constraints.
- Execute only in the NestJS 11 / Node.js 22 BullMQ communications worker on Linux/AMD64 ECS Fargate. Preserve TransactionalMessagePort, the durable PostgreSQL outbox, isolated adapter types, and zero frontend or synchronous API calls to a provider. Source: TCW-71 acceptance criteria; source pack, Architecture and API Contract.
- Apply explicit abortable five-second network deadlines, stable intent-level idempotency, per-tenant/environment concurrency limits, and transient-only exponential retries with jitter bounded to six hours. Source: source pack, Architecture and API Contract; Performance and Reliability Constraints.
- Consume duplicate and out-of-order callbacks safely; acknowledge only after durable enqueue; update operator-visible status within two minutes; publish CloudWatch and OpenTelemetry signals. Source: TCW-71 acceptance criteria; source pack, Architecture and API Contract.

Security requirements

- Use TLS 1.2+, encryption at rest, least privilege, isolated development/staging/production credentials, quarterly credential rotation or short-lived IAM task-role credentials, and auditable configuration changes. Source: source pack, Security and Compliance Requirements.
- Verify cryptographically signed callbacks against the unmodified body, headers and query string; cap ingress at 512 KiB, reject events older than five minutes, pin expected signing certificates/issuers where applicable, and deduplicate provider event IDs. Source: TCW-71; source pack, Architecture and API Contract.
- Do not expose provider credentials, message bodies, customer addresses, identity tokens, payment data, passwords, or full profiles in logs, traces, dashboards, metrics, CI output, or vendor metadata. Use opaque tenant/intent identifiers only. Source: source pack, Security and Compliance Requirements.

Compliance requirements

- Document SOC 2 and GDPR-equivalent controls, an executable data-processing agreement, processing/storage locations for the US and Canada, and a credible future EU-region deployment. Source: TCW-71 business requirements; source pack, Security and Compliance Requirements.
- Support tenant-isolated erasure/suppression, approximately 30-day vendor operational metadata retention, and 13-month provider-neutral RelayForge audit retention. Source: source pack, Security and Compliance Requirements.

Technology Stack

Technology	Detected project implementation and integration implication
Product architecture	Multi-tenant returns, warranty, refund, and shipping-exception workflows; communications isolated behind a provider-neutral domain port.
Frontend	Next.js ^15.3.3, React ^19.1.0, TanStack React Query ^5.80.0; browser must never contact the provider.
Backend and workers	NestJS ^11.1.3 on Express, strict TypeScript ^5.7.3, BullMQ ^5.53.2, ioredis ^5.6.1, Pino ^9.7.0.
Runtime and platform	Node.js 22.15.1 in CI/containers; engine >=22.14.0 and <23; Linux/AMD64 AWS ECS Fargate.
Database and storage	PostgreSQL 16.9 / Amazon RDS Multi-AZ, Prisma ^6.9.0; Redis 7.4 / ElastiCache; S3/KMS signed object links; local MinIO.
Authentication	OIDC/OAuth 2.1 authorization-code with PKCE and tenant-scoped RBAC; AWS ECS task-role IAM is available for provider access.
Deployment and cloud	AWS private subnets, Application Load Balancer, NAT egress, ECR, CloudFront, AWS WAF, CloudWatch, OpenTelemetry; GitHub Actions OIDC and Trivy.
Package/build tooling	pnpm 10.12.1 monorepo, Turborepo ^2.5.4, Vitest ^3.2.0, ESLint ^9.28.0, Prettier ^3.5.3.
Existing integrations	Stripe, Avalara, and Shippo; no transactional-email provider SDK is present.
Infrastructure as code	Dockerfile and Docker Compose are present; deployment documentation describes Terraform, but Terraform configuration files are not present in the pinned repository tree.
Coding style	Strict TypeScript, provider-neutral domain boundaries, dependency injection, linting/formatting, deterministic lockfiles, unit and contract tests.

Pinned implementation evidence: [README](#), [root package configuration](#), [API package configuration](#), [worker package configuration](#), [deployment architecture](#), [service boundaries](#), [container build](#), and [CI workflow](#).

Evaluation Methodology

Repository snapshot: default branch main, commit 8b4c93b20b2949ea95414e81b95fe0f1c10f0fe1. Vendor research as-of: Aug 4, 2026 .

Exactly eight transactional-email platforms were assessed using three independent adoption signals: official GitHub SDK-repository stars, a date-bounded official npm weekly-download snapshot, and inclusion in a G2 category/product comparison. npm links are immutable dated snapshots; GitHub and G2 values were observed on the research as-of date. A separately archived GitHub/G2 capture was not available.

Initial shortlist ranking: feature coverage 40%, technology compatibility 30%, documentation 15%, and estimated plan affordability 15%. The top five are Amazon SES, Twilio SendGrid, Mailgun, Resend, and Postmark; Brevo, Mailjet, and MailerSend remain documented but are not individually reviewed.

Platform	Official SDK stars	npm weekly downloads	Third-party category signal	Initial score / 10
Amazon SES	3,652	31,77,631	G2 · 201 reviews	9.45
Twilio SendGrid	3,054	44,73,623	G2 · 366 reviews	8.85
Mailgun	549	10,13,814	G2 · 312 reviews	8.70
Resend	939	93,01,142	G2 · 9 reviews	7.70
Postmark	360	11,31,305	G2 · 31 reviews	7.35
Brevo	120	2,27,119	G2 · 2,557 reviews	7.15
Mailjet	255	1,55,274	G2 · 173 reviews	7.15
MailerSend	176	81,336	G2 · 46 reviews	6.75

Signal evidence and dated snapshots:

- Amazon SES: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).
- Twilio SendGrid: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).
- Mailgun: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).

- Resend: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).
- Postmark: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).
- Brevo: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).
- Mailjet: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).
- MailerSend: [GitHub SDK snapshot](#); [npm dated weekly snapshot](#); [G2 category comparison](#).

Near-tie rationale: Brevo and Mailjet tie at 7.15. Brevo's broader G2 presence and higher npm usage outweigh Mailjet's higher SDK star count, while Brevo's product listing covers a broader marketing suite rather than exclusively transactional email. AWS SDK stars describe a shared AWS SDK monorepo and are deliberately not treated as SES-only community activity.

Final score = Feature Coverage × 25% + Tech Stack Compatibility × 20% + Integration Complexity × 10% + inverse-normalized monthly cost × 15% + SDK Quality × 10% + Documentation Quality × 10% + Security × 10%. Easy = 10, Medium = 6, Complex = 3; cost score = $1 + 9 \times (\text{highest evaluated cost} - \text{platform cost}) / (\text{highest evaluated cost} - \text{lowest evaluated cost})$. Scalability and Community Support are disclosed separately and do not enter the specified overall-score formula.

The 10,000-MAU estimate assumes one transactional email per active user per month plus all 180 merchant identities. This is a comparison baseline, not the production load; production planning separately uses 35,000 messages/day.

Comparison Matrix

Platform	Feature Coverage (1–10)	Tech Stack Compatibility (1–10)	Integration Complexity	Estimated Monthly Cost (USD)	SDK Quality (1–10)	Documentation Quality (1–10)	Security (1–10)	Scalability (1–10)	Community Support (1–10)	Overall Score
Amazon SES	9	10	Medium	\$1.95	10	9	10	9	9	9.25
Mailgun	9	10	Medium	\$90.00	9	9	10	7	8	7.80
Twilio SendGrid	9	10	Medium	\$89.95	7	10	10	10	9	7.70
Resend	8	8	Easy	\$90.00	9	10	10	3	9	7.65
Postmark	5	8	Easy	\$18.00	8	9	7	2	7	7.50

Security hard gate: Postmark is not eligible for production because its own webhook documentation explicitly says HMAC webhook signature verification is not supported. Resend requires a preapproved increase from its documented default five requests/second before the 25/s normal and 90/s surge requirements can be met.

Recommended Platform

Primary recommendation: Amazon Simple Email Service (Amazon SES), integrated with the SESv2 client and native SES tenant management.

- Native alignment with the current AWS ECS Fargate, IAM task-role, S3, CloudWatch, and OpenTelemetry deployment; no new long-lived vendor sending secret.
- Native isolation for 180 merchants, verified sending identities, tenant-specific configuration sets, independent reputation controls, tenant-level pause/suppression, and regional separation.
- Transparent cost of approximately \$1.95/month for 10,000 comparison-baseline messages and 180 tenants; approximately \$81.75 per 22-business-day month or \$111.15 for a conservative 30-day peak month, before attachments and auxiliary AWS services.
- Documented SOC 2 / GDPR / DPA controls, signed SNS event verification, adjustable sending quotas, and a published 99.9% regional service commitment.

Expected delivery effort: 7–10 engineering days, including a production-quota spike, merchant-domain onboarding automation, adapter/worker integration, signed callback processing, observability, and load validation. Ongoing maintenance: low-to-moderate operational effort for sender reputation, DNS onboarding, quota monitoring, regional configuration, and periodic SDK dependency updates.

Alternatives: Mailgun when a turnkey third-party dashboard, EU API endpoint, and signed HMAC callbacks are preferable; Twilio SendGrid when a formal 99.95% API SLA, very high documented throughput, enterprise subuser administration, and mature analytics justify the higher plan cost.

Individual Platform Research

Amazon SES

Amazon SES is AWS's metered transactional-email service, using the modular SESv2 JavaScript client and SigV4-signed API calls. Its native tenant-management model isolates identities, configuration sets, templates, reputation, and sending state per merchant, making it especially well matched to RelayForge's existing AWS boundary.

Requirement coverage: all six transactional message types; localized HTML/plain-text content; 180 verified sender identities; native tenant isolation and per-tenant pause; attachments or expiring S3 links; configuration-set events; bounce/complaint suppression; sandbox access; region-specific processing. Gap: SendEmail has no documented provider-side idempotency token, so crash-after-submit ambiguity requires application-owned reconciliation and a stable opaque intent tag.

Technology compatibility: Next.js/React — Yes, no client integration; NestJS/Express — Yes, injected SESv2 adapter; Node.js 22/strict TypeScript — Yes; PostgreSQL/Prisma — Yes, unchanged outbox/audit; Redis/BullMQ — Yes, worker-only dispatch; ECS Fargate/Linux AMD64 — Yes; AWS/S3/CloudWatch/OpenTelemetry — Yes; existing OIDC/PKCE/RBAC — Yes, unchanged; IAM task-role authentication — Yes. Every identified stack item is compatible.

SDK and documentation: @aws-sdk/client-sesv2 3.1102.0; latest stable release Aug 3, 2026 ; actively maintained modular TypeScript SDK with official JavaScript, Python, Java, .NET, Go, PHP, Ruby and other AWS SDKs. Official SDK repository and tenant implementation guide provide runnable examples; AWS SDK major-version migration guidance is available. Documentation quality: 9/10.

Integration complexity: Medium — native cloud alignment is excellent, but production quota approval, 180-domain verification, signed SNS ingress, and application-managed idempotency require explicit engineering.

Pricing and maturity: up to 3,000 SES message charges/month during the eligible first 12 months; outbound email \$0.10/1,000, tenant \$0.005/month, and tenant-routed email \$0.005/1,000. At 10,000 emails and 180 tenants: $\$1.00 + \$0.90 + \$0.05 = \$1.95/\text{month}$.

Attachment/message data costs \$0.12/GB; SNS, CloudWatch, NAT, dedicated IPs and optional deliverability features are extra. SES launched in 2011, so it has approximately 15 years in operation; AWS enterprise support is available; SLA 99.9%; API version SESv2 and semantically versioned modular SDK v3; documented successor-version migration path; current SDK release cadence is high.

Authentication and compliance: AWS IAM/SigV4 with short-lived ECS task-role credentials; SMTP credentials are available but not recommended. AWS documents SOC 2, GDPR-compatible processing, an automatic data-processing addendum, encryption, and region-scoped services. Use HTTPS/SNS signature validation and tenant-scoped resource associations.

Advantages: native tenant isolation; IAM without static API keys; lowest transparent operating cost; AWS-region and observability alignment.

Disadvantages: production sandbox/quotas must be increased; 180 merchant DNS identities need onboarding; tenant resources are regional; provider-side send idempotency is not publicly documented.

Risks: launch blocked by unapproved send quotas; duplicate sends after an ambiguous network outcome; poor sender reputation at one merchant; incorrect certificate/topic validation on event ingress.

Implementation steps

1. Create isolated AWS accounts or environment-scoped SES resources and choose approved deployment regions.
2. Request production access plus at least 120 messages/second and 50,000 messages/day; record the approval before launch.
3. Create 180 SES tenants, verified DKIM/SPF/DMARC identities, tenant configuration sets, and tenant-scoped suppression settings.
4. Install the pinned SESv2 JavaScript client through pnpm after dependency/security review.
5. Grant the ECS worker task role only required ses:SendEmail permissions and resource conditions; configure region, deadlines, and environment separation.
6. Implement a NestJS TransactionalMessagePort adapter around the PostgreSQL outbox, stable intent tag, BullMQ worker, and S3 signed document links.
7. Create SES event destinations and signed SNS HTTPS callbacks; validate certificate, topic, timestamp, raw body, size, and duplicate event IDs.
8. Exercise mailbox simulator/sandbox, template locales, merchant disablement, duplicate/out-of-order callbacks, and ambiguous send outcomes.
9. Publish CloudWatch/OpenTelemetry queue, send, throttle, bounce, complaint, latency, and reputation alerts.
10. Deploy through GitHub Actions OIDC/ECS Fargate, load-test 90/s bursts, and roll out tenant-by-tenant.

Public authentication example

```
import { SESv2Client, SendEmailCommand } from "@aws-sdk/client-sesv2";

const client = new SESv2Client({ region: process.env.AWS_REGION }); // ECS task role supplies short-lived credentials
await client.send(new SendEmailCommand({ TenantName: intent.tenantId, FromEmailAddress: intent.sender, Destination: { ToAddresses: [intent.recipient] }, Content: { Simple: { Subject: { Data: intent.subject }, Body: { Text: { Data: intent.text } } } }, ConfigurationSetName: intent.configurationSet, EmailTags: [{ Name: "intent_id", Value: intent.id } ] }, { abortSignal: AbortSignal.timeout(5000) }));
```

Adapted from the official SES tenant sending example; all identifiers are runtime variables, and no credential or customer data is present.

Official evidence

- [Official SDK](#)
- [SES tenant and sending documentation](#)
- [Pricing](#)
- [Sending quotas](#)
- [Signed SNS notification validation](#)

- [AWS SOC 2, GDPR and DPA](#)
- [99.9% service-level agreement](#)
- [SES launch announcement](#)

All vendor pages and SDK/package evidence reflect the research as-of date; a separately archived capture was not available.

Mailgun

Mailgun is a developer-focused transactional-email API with regional US/EU endpoints, per-domain configuration, event webhooks, and maintained official Node.js tooling. It offers stronger out-of-the-box third-party email operations than SES while preserving a backend-only worker architecture.

Requirement coverage: transactional HTML/text templates, attachments, custom metadata, signed HMAC event callbacks, suppression, EU API routing, sandbox, 1,000 sending domains, 30-day event-log retention on Scale, and subaccount isolation. Exact 90/s account throughput and provider-side send idempotency are not publicly documented within the reviewed source cap.

Technology compatibility: Next.js/React — Yes, no frontend dependency; NestJS/Express — Yes; Node.js 22/strict TypeScript — Yes; PostgreSQL/Prisma and Redis/BullMQ — Yes; Linux/AMD64 ECS Fargate and AWS HTTPS egress — Yes; S3 signed links — Yes; existing OIDC/RBAC — Yes, unchanged; vendor API-key authentication — Yes via AWS Secrets Manager; US/EU regional endpoint — Yes.

SDK and documentation: mailgun.js 13.3.0; latest stable release **Jul 9, 2026** ; actively maintained TypeScript/JavaScript SDK with official language support and runnable API examples; official migration handbook available. Documentation quality: 9/10.

Integration complexity: Medium — configure API-key rotation, merchant domains/subaccounts, shared signed HMAC ingress, tenant event mapping, and quota verification.

Pricing and maturity: Free developer tier supports limited sending and one domain; Basic/Free tiers have one domain, Foundation supports 1,000 domains but only five-day logs, and Scale costs \$90/month for 100,000 emails, 1,000 domains and 30-day logs. Scale overage starts at \$1.10/1,000. Comparison-baseline estimate: \$90/month; conservative 1,050,000-email month: approximately \$1,135 before volume discounts. Founded in 2010, approximately 16 years operating; enterprise support and Scale phone/chat support are available. Published uptime SLA, formal API deprecation schedule, and exact send TPS: Not publicly documented. Latest SDK release indicates active maintenance.

Authentication and compliance: API key / HTTP Basic for the API; HMAC-SHA256 signatures over webhook timestamp and token, replay-window validation, optional transport-level certificate checks; official SOC 2 Type I/II and GDPR/CCPA documentation. EU domains use the documented EU API endpoint.

Advantages: 1,000 domains and 30-day logs at a predictable tier; signed callbacks; explicit US/EU routing; strong operational tooling.

Disadvantages: substantially higher subscription cost than SES; static credentials require rotation; 90/s contracted throughput is not published; full message retention can persist up to seven days on Scale.

Risks: insufficient contracted TPS; tenant/domain misrouting across regions; webhook signing-key rotation gaps; customer data accidentally retained in vendor message logs.

Implementation steps

1. Create separate development, staging, and production Mailgun accounts/subaccounts; select the US or EU endpoint.
2. Choose Scale for 1,000 domains and 30-day logs; obtain written 90/s throughput confirmation.
3. Generate scoped API and webhook-signing keys; store and rotate them in AWS Secrets Manager.
4. Install the pinned mailgun.js package with pnpm and verify Node 22/TypeScript compatibility.
5. Verify merchant domains and map each opaque RelayForge tenant to its Mailgun domain/subaccount.
6. Implement the NestJS worker-only TransactionalMessagePort adapter with five-second deadlines and opaque message metadata.
7. Register shared HTTPS event callbacks and verify HMAC timestamp/token/signature before durable enqueue.
8. Test sandbox domains, fr-CA/English templates, expiring S3 links, bounce suppression, retry classification, and duplicate callbacks.
9. Emit redacted CloudWatch/OpenTelemetry delivery, latency, throttle, and tenant-reputation signals.
10. Deploy through ECS Fargate, verify regional data routing, load-test surge capacity, and enable tenants progressively.

Public authentication example

```
import Mailgun from "mailgun.js"; import FormData from "form-data";  
const client = new Mailgun(FormData).client({ username: "api", key: process.env.MAILGUN_API_KEY!, url: process.env.MAILGUN_API_URL  
});
```

Official Mailgun client-authentication pattern; credentials are referenced only through environment variables.

Official evidence

- [Official SDK](#)
- [API reference and US/EU endpoints](#)
- [Pricing](#)
- [Signed webhook verification](#)
- [SOC 2 and GDPR compliance guide](#)
- [Migration guide](#)
- [Company history](#)

All vendor pages and SDK/package evidence reflect the research as-of date; a separately archived capture was not available.

Twilio SendGrid

Twilio SendGrid is a mature transactional-email platform with API v3, dynamic templates, merchant subusers, signed event callbacks, regional EU endpoints, and enterprise deliverability tooling. Its published API SLA and very high service throughput are particularly attractive when contractual availability matters.

Requirement coverage: transactional templates and attachments, merchant subusers, signed event webhooks or OAuth 2.0 webhook security, sandbox, EU regional subusers, bounce/complaint processing, and 180-domain administration. Seven-day event activity is included on Pro; reaching the 30-day metadata target requires a separately priced add-on.

Technology compatibility: Next.js/React — Yes, no browser access; NestJS/Express and Node.js 22/TypeScript — Yes; PostgreSQL/Prisma and BullMQ/Redis — Yes; AWS ECS Fargate/Linux and HTTPS/NAT — Yes; existing OIDC/tenant RBAC — Yes, unchanged; scoped vendor API keys in Secrets Manager — Yes; US/EU regional endpoint — Yes.

SDK and documentation: @sendgrid/mail 8.1.6; latest stable release Sep 19, 2025 ; official clients in Node.js, Python, Java, Go, Ruby, PHP, and C#. Release cadence of the current Node send package is slower than AWS/Resend/Mailgun, despite broad adoption. Full quickstarts, API reference, runnable seven-language examples, and API v2-to-v3 migration guidance support a 10/10 documentation rating.

Integration complexity: Medium — use Pro subuser management, authenticate 180 merchant domains, configure signed shared callbacks, add 30-day event-history coverage, and constrain all custom arguments to opaque identifiers.

Pricing and maturity: free trial is 100 emails/day for 60 days; Essentials starts at \$19.95/month but lacks merchant subuser administration. Pro starts at \$89.95/month for 100,000 emails, subuser management, five event-webhook endpoints, and seven-day activity history; 30-day history is an additional purchase with pricing not publicly documented. Comparison-baseline estimate: at least \$89.95/month plus the 30-day-history add-on. Enterprise support is available. Standard API SLA is 99.95%, rising to 99.99% for specified paid support packages; the official agreement describes up to 15,000 North American API requests/second under its conditions. API v3 and official v2-to-v3 migration/deprecation guidance are documented. Exact vendor founding year: Not publicly documented within the selected official source set.

Authentication and compliance: scoped bearer API keys; signed Event Webhook verification and optional OAuth 2.0 on callback delivery; SOC 2 Type II, GDPR, enforced TLS, and EU data residency publicly documented. SendGrid says categories/custom arguments are classified Not PII and may persist, so send only opaque intent/tenant tokens, never customer identifiers.

Advantages: published 99.95% API SLA and high documented throughput; mature seven-language SDK ecosystem; enterprise subusers and domain administration; signed events plus EU residency.

Disadvantages: \$89.95 minimum for required subusers; 30-day history add-on is not publicly priced; Node SDK latest stable release is substantially older than peers; only five shared webhook endpoints on Pro.

Risks: confidential data embedded in persistent custom arguments; underestimated event-history add-on; incorrect subuser isolation; merchant warm-up or reputation issues.

Implementation steps

1. Create isolated accounts or environment subusers and select US or EU regional processing.
2. Enable a Pro-or-higher plan, provision merchant subusers, and verify 180 sending domains.
3. Generate minimally scoped sending and webhook credentials; store with dual-key rotation in Secrets Manager.
4. Install the pinned `@sendgrid/mail` package with `pnpm` and configure a NestJS worker-scoped provider.
5. Implement opaque tenant/intent metadata, localized HTML/text rendering, S3 signed links, and a five-second deadline.
6. Configure signed Event Webhook or OAuth 2.0, preserving raw bodies and validating timestamp/replay limits.
7. Keep categories and custom arguments strictly opaque because SendGrid treats them as persistable Not PII fields.
8. Test sandbox mode, template variants, callback deduplication, permanent-vs-transient errors, and subuser disablement.
9. Add the 30-day activity-history option and CloudWatch/OpenTelemetry delivery and queue monitoring.
10. Deploy via ECS Fargate and perform authenticated 90/s load, regional routing, and merchant warm-up validation.

Public authentication example

```
import sgMail from "@sendgrid/mail";  
sgMail.setApiKey(process.env.SENDGRID_API_KEY!); await sgMail.send(intent.providerMessage);
```

Official SendGrid bearer-key authentication pattern; no actual credential or recipient is shown.

Official evidence

- [Official SDK](#)
- [Mail Send v3 documentation](#)
- [Pricing, certifications and EU residency](#)
- [Signed webhook security](#)
- [Event metadata privacy limitations](#)
- [API rate limits](#)
- [99.95% API SLA and throughput commitment](#)
- [API v2-to-v3 migration guide](#)

All vendor pages and SDK/package evidence reflect the research as-of date; a separately archived capture was not available.

Resend

Resend is a modern transactional-email API with first-class TypeScript tooling, React Email support, signed webhooks, explicit provider-side idempotency, and a strong current npm footprint. Its implementation experience is excellent, but documented default API throughput and US data processing do not meet launch requirements without contractual changes.

Requirement coverage: localized HTML/text rendering, attachments, up to 1,000 verified domains on Scale, 30-day retention, signed callbacks, sandbox/test addresses, and 24-hour provider-side Idempotency-Key support. Hard capacity gap: the default account-wide limit is five requests/second versus 25/s normal and 90/s peak; an approved increase or carefully designed batch workflow is mandatory.

Technology compatibility: Next.js/React and React Email — Yes, provider remains worker-only; NestJS/Express, Node.js 22 and strict TypeScript — Yes; PostgreSQL/Prisma and Redis/BullMQ — Yes; ECS Fargate/Linux and AWS HTTPS egress — Yes; existing OIDC/tenant RBAC — Yes, unchanged; scoped API key — Yes; future EU data residency — Partial because the privacy policy discloses US processing despite multi-region delivery.

SDK and documentation: resend 6.18.1; latest stable release **Jul 28, 2026** ; official SDKs in Node.js, PHP, Python, Ruby, Go, Rust, Java and .NET; complete runnable examples, typed idempotency examples, API reference, signed webhook verification, and migration guides. Documentation quality: 10/10.

Integration complexity: Easy — typed Node SDK, documented signed callbacks, and built-in idempotency keep implementation small, provided a written rate-limit increase is obtained.

Pricing and maturity: Free includes 3,000 emails/month, 100/day, one domain, and 30-day retention; Pro costs \$20/month for 50,000 emails and 10 domains; Scale costs \$90/month for 100,000 emails, 1,000 domains, and 30-day retention, with \$0.90 per 1,000 excess emails. The 180-domain requirement therefore requires Scale: \$90/month at the comparison baseline and approximately \$945 for 1,050,000 messages. Founded in 2023, approximately three years operating; Scale/Enterprise support and published support-response SLA exist; service-availability SLA, API deprecation schedule and guaranteed 90/s quota are Not publicly documented.

Authentication and compliance: scoped bearer API keys; signed Svix-style webhooks verified against the raw payload; documented SOC 2 Type II and GDPR. Privacy policy states personal data is transferred to and processed in the United States, so GDPR contractual controls are not equivalent to EU data residency.

Advantages: first-class typed developer experience; native 24-hour provider-side idempotency; 30-day retention and signed callbacks; 9.3 million weekly npm downloads.

Disadvantages: default five-request/second account limit; US processing undermines future EU residency; 180 domains require \$90 Scale; shorter operating history.

Risks: launch misses 25/s and 90/s requirements without quota approval; EU privacy rollout blocked; shared account-wide rate limit causes tenant contention; accidental customer identifiers in persistent tags.

Implementation steps

1. Create separate environment accounts; select Scale because 180 domains exceed the Pro plan.
2. Obtain a written API-quota increase above 90 requests/second before production.
3. Create scoped API and signing keys and configure automated Secrets Manager rotation.

4. Install the pinned resend SDK using pnpm after lockfile and dependency review.
5. Verify merchant domains and map each opaque tenant to an approved sender identity.
6. Implement the NestJS BullMQ adapter with provider Idempotency-Key, a five-second timeout, and application-owned outbox state.
7. Configure and verify signed raw-body webhooks; enforce event replay, size, tenant, and duplicate-ID controls.
8. Test sandbox addresses, templates, 24-hour idempotency, Retry-After handling, and per-tenant disablement.
9. Publish CloudWatch/OpenTelemetry queue, throttle, failure, and data-retention signals.
10. Deploy to ECS only after rate-limit and regional-processing/privacy approvals.

Public authentication example

```
import { Resend } from "resend"; const client = new Resend(process.env.RESEND_API_KEY);  
await client.emails.send(intent.providerMessage, { idempotencyKey: intent.id });
```

Official Resend authentication and idempotency pattern; no customer data or secret value is included.

Official evidence

- [Official SDK](#)
- [Send-email API reference](#)
- [Pricing and SOC 2/GDPR disclosures](#)
- [Default five-request-per-second limit](#)
- [Signed webhook verification](#)
- [US processing disclosure](#)
- [Provider-side idempotency](#)
- [Migration guide and company history](#)

All vendor pages and SDK/package evidence reflect the research as-of date; a separately archived capture was not available.

Postmark

Postmark specializes in transactional email with straightforward Node.js integration, strong sending-domain ergonomics, message streams, and flexible retention. It is inexpensive for 180 merchant domains but fails RelayForge's explicit requirement for cryptographically signed webhooks.

Requirement coverage: HTML/text templates, attachments, tenant sender domains, message streams, sandbox, delivery/bounce callbacks, configurable retention, and tenant metadata. Critical gap: Postmark's own webhook documentation explicitly states that HMAC webhook signature verification is unsupported; HTTP Basic authentication and IP allowlisting are not a substitute for the required cryptographic signature. Processing is US-based.

Technology compatibility: Next.js/React — Yes, no frontend dependency; NestJS/Express and Node.js 22/TypeScript — Yes; PostgreSQL/Prisma and Redis/BullMQ — Yes; AWS ECS Fargate/Linux — Yes; existing OIDC/tenant RBAC — Yes; API token via Secrets Manager — Yes; future EU regional processing — Partial because subprocessors and processing are US-based.

SDK and documentation: postmark 5.1.0; latest stable release **Jul 10, 2026** ; maintained official Node.js library plus other official client-language libraries; API quickstarts, runnable samples, sender-signature administration, and migration guidance are available. Documentation quality: 9/10. The npm stable version is newer than the GitHub release-page tag, so the registry is the authoritative release source.

Integration complexity: Easy — sender tokens and domain APIs are simple, but the signed-webhook compliance gap cannot be solved by integration effort alone.

Pricing and maturity: free Developer tier permits 100 emails/month; at the 10,000-email price selector, Basic is \$15/month for five domains, Pro \$16.50 for ten domains, and Platform \$18/month for unlimited custom domains. Therefore 180 merchants require the \$18 Platform plan; 45-day default content history must be explicitly shortened to 30 days. Platform overage is \$1.20/1,000; a 1,050,000-email month is approximately \$1,266. Enterprise-level uptime SLA, founding year, API version/deprecation schedule, and account-level send TPS: Not publicly documented. Support and an actively maintained Node SDK are documented.

Authentication and compliance: server API token and separate account-management token; webhook HTTP Basic authentication plus IP allowlisting, but no HMAC signing. GDPR/DPA and SCCs are documented; only the underlying data center's SOC 2 Type II accreditation is publicly established in the reviewed sources, not an equivalent vendor-wide certification.

Advantages: \$18 unlimited-domain plan; low integration effort; maintained typed SDK; configurable 7–365-day retention.

Disadvantages: no cryptographic webhook signatures; US-only processing; default 45-day full-message history; no published per-account TPS or availability SLA.

Risks: unauthenticated or replayed callback tampering; failure of future EU-residency review; excessive full-body retention; undocumented surge throughput.

Implementation steps

1. Create isolated Postmark environments and select Platform for unlimited merchant domains.
2. Register merchant sending domains, DKIM signatures, message streams, and account/server tokens.
3. Store and rotate separate environment API credentials through AWS Secrets Manager.
4. Install the pinned postmark Node.js SDK with pnpm and wire a worker-only NestJS adapter.
5. Configure reviewed localized templates, opaque intent identifiers, expiring S3 links, and five-second deadlines.
6. Reduce the default 45-day content history to the required 30-day retention setting.
7. Evaluate webhook HTTP Basic plus IP allowlisting, and record that these do not satisfy the signed-callback requirement.
8. Test sandbox sending, callback ordering/deduplication, suppression, and merchant disablement.
9. Add CloudWatch/OpenTelemetry monitoring and confirm real sustained/surge sending limits.

10. Do not deploy to production unless the vendor supplies documented cryptographic webhook signatures.

Public authentication example

```
import { ServerClient } from "postmark"; const client = new ServerClient(process.env.POSTMARK_SERVER_TOKEN!);  
await client.sendEmail(intent.providerMessage);
```

Official Postmark server-token authentication pattern; no credential or customer data is included.

Official evidence

- [Official SDK](#)
- [API reference](#)
- [Pricing and unlimited sending domains](#)
- [Webhook security limitations](#)
- [GDPR and US subprocessors](#)
- [Data-center security posture](#)
- [Official client libraries and samples](#)
- [Sender-signature management](#)

All vendor pages and SDK/package evidence reflect the research as-of date; a separately archived capture was not available.

Architecture Blueprint

High-Level Integration Architecture

- Keep Next.js and the synchronous NestJS case API provider-neutral: case mutations commit business state and a communication intent to PostgreSQL in the same transaction.
- Dispatch intent IDs through the existing Redis/BullMQ communications queue to a dedicated NestJS worker running in private-subnet ECS Fargate.
- Resolve merchant configuration and invoke SESv2 through an injected TransactionalMessagePort adapter authenticated only by its task role.
- Route SES configuration-set delivery signals through a dedicated SNS topic to a narrowly scoped HTTPS ingress and asynchronous event processor.
- Persist normalized tenant/message/event state in PostgreSQL; surface only approved operator status through existing APIs.

Data Flow

- Business action writes case state plus a unique outbox intent containing template key, locale, opaque tenant/intent IDs, and an approved recipient reference.
- Worker claims the intent transactionally, checks tenant enablement/suppression, renders reviewed HTML/plain-text content, and prefers a short-lived S3 URL over retained attachments.
- SESv2 SendEmail receives TenantName, associated configuration set, verified sender, localized content, and an opaque intent tag under a five-second abortable deadline.
- Persist the SES MessageId and provider-neutral accepted state; signed SNS delivery/bounce/complaint events are verified and durably queued.
- Idempotent event workers reconcile intent and provider IDs, tolerate stale/out-of-order events, update case history, and expose redacted delivery outcomes.

Authentication Flow

- Retain existing OIDC/OAuth 2.1 PKCE and tenant RBAC for customer/operator access; no browser receives vendor credentials.
- Assign the communications ECS task a dedicated IAM role restricted to SESv2 SendEmail, approved configuration sets, tenant identities, and the allowed region.
- Use AWS default credential resolution and SigV4 signing; rely on automatically rotated short-lived task-role credentials instead of static API keys.

- Verify SNS message signatures with an AWS-controlled HTTPS SigningCertURL, allowlisted topic ARN, trusted certificate chain, and fresh event timestamp.
- Use distinct task roles, SNS topics, ingress URLs, and AWS resources for development, staging, and production.

Error Handling Strategy

- Map validation errors, unverified identities, disabled merchants, authorization failures, and permanent suppressions to terminal domain errors and a redacted dead-letter queue.
- Treat timeouts, connection resets, throttling, documented transient 5xx responses, and temporary delivery failures as retryable.
- Record stable opaque intent/provider/event identifiers and a normalized error class; never persist message bodies, raw addresses, access keys, or full webhook payloads in logs.
- On an ambiguous post-send timeout, mark the intent reconciliation-required rather than automatically creating a second logical customer message.
- Reject invalid signatures, wrong SNS topics, stale timestamps, oversized payloads, and unknown events before they can mutate case state.

Retry Mechanism

- Use a unique PostgreSQL intent constraint and atomic claim/lease so concurrent BullMQ attempts cannot dispatch the same logical message simultaneously.
- Apply capped exponential backoff with jitter only to classified transient failures, with a hard six-hour total retry horizon.
- Honor provider backoff signals, preserve one stable intent correlation tag across attempts, and pause/reconcile outcomes when request completion is ambiguous.
- Deduplicate inbound callbacks by provider plus event identifier and apply terminal-state precedence to out-of-order transitions.
- Move exhausted or nonretryable jobs to a tenant-scoped dead-letter queue with redacted diagnostic metadata.

Rate-Limit Handling

- Request SES production access, at least 120 sends/second, and at least 50,000 sends/day before release; sandbox defaults of 200/day and 1/second are insufficient.
- Apply a Redis-backed global token bucket plus merchant-level fairness limits so one tenant cannot consume all account capacity.
- Scale communications worker concurrency from queue depth while respecting approved SES account/region quotas and the five-second request deadline.
- On throttling, reduce concurrency, add jitter, monitor time-to-provider-handoff, and preserve the 60-second p95 / five-minute p99 service objectives.

- Run a ten-minute authenticated 90/s surge test and alarm before sustained quota exhaustion.

Caching Recommendations

- Cache only non-sensitive tenant enablement, region, configuration-set name, verified sender identity, and template version with short TTLs and explicit invalidation.
- Use Redis for rate-limit counters, lightweight dispatch leases, and suppression-cache hints; PostgreSQL remains authoritative.
- Cache validated AWS signing certificates for a bounded interval while rechecking issuer, hostname, and topic constraints.
- Never cache recipient addresses, rendered bodies, credentials, long-lived signed object URLs, or raw webhook content outside approved encrypted stores.

Security Considerations

- Enforce TLS 1.2+, encryption at rest, least-privilege IAM, environment isolation, merchant resource association, and auditable domain/template changes.
- Preserve the exact SNS body, enforce the 512 KiB limit, validate signature/certificate/topic before JSON trust, and reject timestamps older than five minutes.
- Use opaque metadata only; payment information, passwords, identity tokens, complete profiles, and confidential customer fields must never enter provider tags or logs.
- Prefer short-lived signed S3 links and explicitly bound provider-side operational metadata to the 30-day target where configurable.
- Apply 13-month partitioned provider-neutral audit retention, tenant-scoped erasure/suppression, and per-region residency reviews.

Monitoring & Logging

- Publish redacted CloudWatch/OpenTelemetry metrics for enqueue-to-handoff latency, acceptance, delivery, deferral, bounce, complaint, rejection, and callback processing.
- Track tenant fairness, provider quota utilization, queue depth/age, retry counts, dead-letter volume, credential/role failures, and merchant-domain verification.
- Alert on 25/s steady-state and 90/s burst saturation, p95 handoff above 60 seconds, p99 above five minutes, and callback-to-operator delay above two minutes.
- Propagate trace/span and opaque intent IDs across the API, outbox, worker, SES tags, SNS events, and audit pipeline without logging PII.
- Create merchant-level bounce/complaint reputation dashboards and circuit breakers that pause only the affected tenant.

Deployment Considerations

- Build the communications adapter into the existing Node 22 / pnpm / Turborepo container for Linux/AMD64 ECS Fargate with no persistent local disk.
- Deploy through existing GitHub Actions OIDC, container scanning, ECR, and environment-specific ECS services in private subnets with managed NAT.
- Provision SES identities, tenant associations, configuration sets, SNS topics, callback routes, CloudWatch alarms, and task roles as reviewed infrastructure configuration.
- Use SES mailbox simulator/nonproduction resources in local and staging tests; local Docker Compose must never contact production provider resources.
- Roll out by merchant cohort with feature flags, contract tests, DNS readiness gates, surge load tests, and an immediate per-tenant pause path.
- Replicate identities, configuration sets, suppression controls, and tenant resources explicitly when a future approved EU region is added.

Risks & Mitigation

Risk	Impact	Mitigation
SES account remains sandboxed or approved throughput is below 90/s	Launch throughput and handoff SLOs fail.	Secure production access and $\geq 120/s$ / $\geq 50,000$ -day quotas; run a ten-minute 90/s preproduction test.
Crash or timeout after SES accepts a send	A customer can receive a duplicate logical message.	Use a unique outbox claim, stable opaque intent tag, provider MessageId reconciliation, and a quarantine state for ambiguous outcomes.
Merchant DNS or reputation configuration is incorrect	One merchant's delivery fails or reputation degrades.	Automate DKIM/SPF/DMARC verification, isolate SES tenants/configuration sets, alarm per merchant, and pause only the affected tenant.
SNS verification, certificate allowlist, or event ordering is implemented incorrectly	Spoofed, stale, replayed, or out-of-order callbacks corrupt case history.	Validate raw signature/certificate/topic/timestamp, enforce 512 KiB, deduplicate event IDs, and use monotonic/terminal-state rules.
Regional resources or metadata leak across tenant/privacy boundaries	Privacy, residency, and retention commitments are breached.	Use region-specific tenant resources, opaque metadata, short-lived S3 links, encrypted stores, strict role conditions, and retention/erasure tests.

Cost Analysis

Comparison model: 10,000 MAUs correspond to one email per user/month, and all 180 merchant sender identities are enabled. Taxes, SMS/marketing features, optional support contracts, and existing application-hosting costs are excluded.

Platform / required plan	10,000-email + 180-tenant baseline	Production planning at 1,050,000 emails	Critical pricing caveat
Amazon SES + native tenants	\$1.95/month	\$111.15/month	Add message/attachment bytes at \$0.12/GB plus SNS, CloudWatch, NAT, and any optional dedicated IPs.
Mailgun Scale	\$90.00/month	Approximately \$1,135/month	Scale is required for both 1,000 domains and 30-day logs; volume contracts may reduce overage.
Twilio SendGrid Pro	At least \$89.95/month	Volume-tier quote required	Seven-day activity is included; the required 30-day event-history add-on has no publicly documented price.
Resend Scale	\$90.00/month	Approximately \$945/month	180 domains require Scale; default API limit is only five requests/second until support approves an increase.
Postmark Platform	\$18.00/month	Approximately \$1,266/month	Unlimited domains are included, but signed callbacks are unavailable and default full-message retention is 45 days.

Amazon SES arithmetic: $10,000 \times \$0.10/1,000 = \1.00 ; $180 \times \$0.005 = \0.90 ; $10,000 \times \$0.005/1,000 = \0.05 ; total \$1.95/month. At 35,000 messages/day for 22 business days: 770,000 messages produce $\$77.00 + \$0.90 + \$3.85 = \81.75 /month. A conservative 30-day month produces 1,050,000 messages: $\$105.00 + \$0.90 + \$5.25 = \111.15 /month.

Long-term maintenance is expected to remain low-to-moderate because IAM task roles remove static sending secrets, but merchant DNS/reputation onboarding, quota surveillance, per-region configuration, signed callback validation, and regular SDK upgrades remain recurring responsibilities.

Implementation Checklist

- ☐ Approve SES as the sole primary transactional email provider and keep Mailgun / Twilio SendGrid as documented fallback options.
- ☐ Obtain approved SES production access, ≥ 120 sends/second, and $\geq 50,000$ messages/day.
- ☐ Choose approved US/Canada deployment region and document the future EU-region replication/data-processing plan.
- ☐ Create environment-separated IAM task roles, SES configuration sets, signed SNS topics, callback endpoints, and CloudWatch alarms.
- ☐ Provision and verify all 180 merchant sender domains; configure DKIM, SPF, DMARC, SES tenant associations, and tenant-specific suppression.
- ☐ Pin @aws-sdk/client-sesv2, update the pnpm lockfile deterministically, and complete dependency/security review.
- ☐ Define provider-neutral TransactionalMessagePort request/result/error contracts and the durable unique-intent PostgreSQL schema.
- ☐ Implement worker-only SESv2 dispatch with task-role authentication, a five-second abortable deadline, opaque metadata, and an ambiguous-outcome reconciliation path.
- ☐ Implement reviewed en and fr-CA HTML/plain-text templates, merchant branding, safe substitutions, and expiring S3 signed document links.
- ☐ Verify raw SNS signatures, signing-certificate issuer/hostname, topic ARN, body-size cap, replay age, deduplication, and out-of-order state handling.
- ☐ Configure Redis/BullMQ global and merchant-level rate limits, transient-only exponential retries with jitter, six-hour cutoff, and a redacted dead-letter queue.
- ☐ Enforce 30-day provider metadata / 13-month provider-neutral audit retention, tenant-scoped erasure, and PII-safe observability.
- ☐ Run sandbox/contract tests, duplicate-delivery simulations, credential/region failure tests, and a full ten-minute 90/s load test.
- ☐ Deploy through GitHub Actions OIDC and ECS Fargate, monitor tenant reputation and handoff SLOs, then release by merchant cohort.

Final Recommendation

Proceed with Amazon SES as the single primary platform. It is the strongest architectural fit for the existing AWS/Node.js/NestJS/BullMQ deployment, is the only reviewed option with native low-cost isolation for all 180 merchants, documents SOC 2/GDPR-compatible controls and signed event verification, and yields the highest weighted score at 9.25/10.

Fund a focused 7–10 engineering-day implementation, beginning with a quota/idempotency technical spike. Use Mailgun when regional third-party operations and turnkey email tooling matter most; use Twilio SendGrid when contractual API availability, enterprise operations, and proven very-high-throughput sending outweigh its higher cost.

Do not approve production until SES production quota, merchant-domain authentication, signed callback verification, duplicate-send reconciliation, regional data handling, 90/s surge capacity, and PII-safe retention/observability have been demonstrated.

